

Part A -- Min-Max Continual-RL Consolidation (experts NOT stored)

Config selector : ppo.method = constrained
Source code : `crl/ppo_continual.py` (PP0AlternationTrainer.run,
 `_train_first_task`, `_constrained_task`, `_evaluate_row`)
 `crl/ppo/trainer.py` (LocalTrainer.train, GlobalTrainer.train,
 PP0Trainer.optimize_batches)
 `crl/duals/controllers.py` (ProjectedAscentDual.update)
Math source : docs/Updated_Objective_for_CRL.pdf, Part A, eqs 1-48.

Idea

Tasks $1..K$ arrive in sequence. A single shared-trunk, multi-head GLOBAL policy $\pi_G = \pi_\phi$ must keep performing on every task it has seen. Nothing about the past is stored except the current parameters ϕ : past-task quantities are re-estimated by FRESH rollouts in the old environments each time they are needed (replay-free, doc p.1 note after eq 3). NO per-task expert model is saved -- this is Part A. The current task's floor is supplied on-the-fly by a throwaway LOCAL policy trained from scratch on that task.

Each stage $k \geq 2$ runs a three-checkpoint local-global cycle (eq 4):

π_G^-	--->	π_L	--->	π_G^+
(frozen		(local		(consolidated
previous		expert		new global,
global)		of task k)		the trainable ϕ)

- * $\pi_G^- = \pi_{\{\phi^-\}}$: previous global, carried in, held FIXED all stage.
- * $\pi_L = \pi_{\{\theta^*\}}$: local policy, trained by plain PP0 on task k only, from $\theta^-(0) = \phi^-$ (eqs 5, 19); then FROZEN. Its value V_k^L is the current-task floor / reference.
- * $\pi_G^+ = \pi_{\{\phi^*\}}$: consolidated global, produced by maximizing the past-task retention objective (eq 25) subject to the one-sided current-task floor $F_k \leq \epsilon$ (eqs 26, 27), enforced as a primal-dual saddle over (ϕ, μ) (eqs 28, 29). At stage end $\phi^- \leftarrow \phi^*$ and task k joins the past set (chaining eq 6).

Only the ACTOR carries the continual-learning constraint. The critic, GAE, value-MSE and entropy bonus are ordinary PP0 on the shared trunk (doc 03).

Inputs

family : ordered K tasks (Atari games), each with a greedy threshold.
global_policy : shared-trunk, per-task actor+critic heads = π_ϕ .

```

cfg.cycles_per_task          : local+global cycles per task (usually 1).
ppo.task1_iters, local_iters, global_iters
ppo.clip_ratio, lr, vf_coef, ent_coef, gae_lambda, n_envs, n_steps, ppo_epochs,
    num_minibatches, max_grad_norm, constraint_episodes, constraint_every,
    past_task_sampling
dual mu controller (ProjectedAscentDual): init, step eta = lr, max_value, eps.
seed.

```

Outputs

```

global_policy after all K tasks    -> final_policy.pt (parameters  $\phi^*$ )
eval_matrix : lower-triangular greedy-score forgetting matrix -> eval_matrix.json
per-phase logs (probe / global / global_diag / gaps / eval rows).

```

Top level (PPOAlternationTrainer.run, method == "constrained")

```

_train_first_task():                # STAGE 1 = plain PPO
    LocalTrainer.train(global_policy, task_1, num_iters = task1_iters)
    # standard PPO trains the GLOBAL net itself; no past tasks, no floor, no mu.
row_1 <- _evaluate_row(1); append to eval_matrix; log

for k = 2 .. K:
    _constrained_task(k)             # local-global cycle
    row_k <- _evaluate_row(k); append to eval_matrix; log
save eval_matrix, final_policy

```

_constrained_task(k)

```

task_k      <- family.tasks[k-1]
past_tasks  <- family.tasks[0 .. k-2]          # tasks 1..k-1
for cycle = 0 .. cfg.cycles_per_task - 1:

    # ----- LOCAL PHASE : produce the frozen floor  $V_k^L$  (eqs 5, 18-23) -----
    local_policy <- clone_policy(global_policy, trainable=True) #  $\theta^0 = \phi^L$ 
    LocalTrainer.train(local_policy, task_k, num_iters = local_iters,
        seed = seed + 1000*k + 13*cycle)                # plain PPO
    frozen_local <- clone_policy(local_policy, trainable=False) # freeze  $\pi_L$ 
    ref_current  <-  $V_k^L = \_eval\_value(frozen\_local, task\_k)$  # eqs 3, 8
        # on-policy STOCHASTIC discounted return, constraint_episodes
        # rollouts. This scalar is the current-task floor.

    # ----- GLOBAL PHASE : constrained consolidation (eqs 25-48) -----
    mu_ctrl.reset()                # fresh dual per global phase
                                    # (warm_start=False -> mu <- init)
    omega <- [1/k] * (k-1)          # uniform past weights  $\omega_i = 1/k$ 
    GlobalTrainer.train(global_policy, task_k, past_tasks,

```

```

        ref_current =  $V_k^L$ , mu_ctrl, omega, eps,
        num_iters = global_iters,
        seed = seed + 1000*k + 13*cycle,
        local_policy = frozen_local) # only for KL/BC diagnostics
log {"phase":"gaps", task=k, cycle,  $V_k^{\text{ref\_local}} = V_k^L$ }
# After the phase, global_policy =  $\pi_G^+$  ; it becomes  $\pi_G^-$  of stage k+1 (eq 6).

```

LocalTrainer.train (the local player = plain PPO on ONE task; eqs 18-23)

```

opt      <- Adam(trainable params, lr, eps=1e-5)
collector <- RolloutCollector(task, n_envs, n_steps, seed) # one stream
met <- 0
for it = 0 .. num_iters-1:
    batch <- collector.collect(policy, gae_lambda) # fresh rollout+GAE
    optimize_batches(policy, opt, streams=[batch], actor_coeffs=[1.0]) # doc 03
    probe(...) # retention log
    gscore <- greedy score every stop_eval_every iters (>= min_iters), else None
    met <- met+1 if gscore >= threshold else 0
    if met >= patience: EARLY STOP
# stop when num_iters exhausted OR greedy threshold met `patience` times.
# actor_coeffs == [1.0] => coeff-sum normalization below is a no-op here.

```

GlobalTrainer.train (the constrained global player; eqs 25-48)

Initialized quantities:

```

opt <- Adam(trainable params of global_policy) # ascends  $L_G$  (eq 43)
past_collectors[i] <- RolloutCollector(past_task_i, seed+101+i) # replay-free
cur_collector <- RolloutCollector(task_k, seed+7)
mu <- mu_ctrl.value ; shortfall <- 0 ; constraint <- 0 ;  $V_k^G$  <- NaN ; met <- 0

```

```

for it = 0 .. num_iters-1:

```

```

# ---- (1) fresh rollouts, past + current, every iteration (replay-free) ----

```

```

if past_task_sampling == "sample" and past_collectors:

```

```

    pick one random past task j

```

```

    past_batches <- [ collect(global, past_task_j) ]

```

```

    past_coeffs <- [ omega[j] * n_past ] # unbiased estimator of full sum

```

```

else:

```

```

    past_batches <- [ collect(global, past_task_i) for all i ]

```

```

    past_coeffs <- omega[:len(past_tasks)] # each = 1/k

```

```

cur_batch <- cur_collector.collect(global, gae_lambda)

```

```

# ---- (2) SLOW dual timescale: refresh floor gap + mu every constraint_every ----

```

```

if it % max(1, constraint_every) == 0:

```

```

     $V_k^G$  <- fresh on-policy Monte-Carlo value of the GLOBAL on task_k # eq 42
    (evaluate_value_and_score, constraint_episodes rollouts)

```

```

    shortfall <- max(0, V_k^L - V_k^G)          # [ V_k^L - V_k^phi ]_+
    constraint <- shortfall * shortfall          # F_k = [V_k^L - V_k^G]_+^2 (eqs 26, 48)
    mu <- mu_ctrl.update(constraint, eps)        # projected dual ASCENT (eq 47):
    # mu <- clip( mu + eta*(F_k - eps), 0, mu_max )
# Between refreshes shortfall / constraint / mu are held fixed (two-timescale).

coeff_k <- mu * 2.0 * shortfall                # differentiated hinge 2 mu [.]_+ (eqs 38, 40)

if diagnostics and it % diag_every == 0: _log_diagnostics(...) # KL(pi_L||pi_G),
    # per-task grad alignment cos(g_new, g_i), V-gap, |g_new|/|g_old|, etc.

# ---- (3) one PPO update over ALL streams (past first, current last) ----
streams <- past_batches + [ cur_batch ]
coeffs <- past_coeffs + [ coeff_k ]
    # per-past-task actor weight = omega_i -> ascends eq 25 term
    # current-task actor weight = 2 mu shortfall -> enforces floor (eq 40)
optimize_batches(global, opt, streams, coeffs, bc = optional_KL) # doc 03

probe("global", k)
gscore <- greedy score of global on task_k (early-stop check)
met <- met+1 if gscore >= threshold else 0
if met >= patience: EARLY STOP
# stop when global_iters exhausted OR global reaches task_k greedy threshold
# `patience` times (past tasks are held up by the omega retention terms).

```

PPOTrainer.optimize_batches (shared core; per-stream actor coefficient)

```

n <- N = n_envs * n_steps (rows per stream, all streams equal length)
mb_size <- N // num_minibatches

# --- coeff-sum normalization (faithful practical DEVIATION #1) ---
# The raw current-task coefficient 2 mu shortfall is UNBOUNDED in mu. Left
# un-normalized, a large mu makes the actor gradient dominate the single
# global grad-norm clip and STARVES the shared critic's value-loss gradient,
# breaking GAE and stalling improvement. Dividing every actor coefficient by
# their sum is a POSITIVE rescaling of the ascent direction: it preserves the
# relative past/current weighting (hence the primal-dual fixed point of eq 29),
# only bounding the step magnitude. For the local phase (coeffs=[1.0]) it is a
# no-op.
coeff_sum <- sum(actor_coeffs)
if coeff_sum > 0: actor_coeffs <- [ c / coeff_sum for c in actor_coeffs ]

for epoch = 1 .. ppo_epochs:
    for each minibatch idx of the shared permutation:
        total_loss <- 0
        for (coeff, batch) in zip(actor_coeffs, streams):

```

```

dist, value <- policy.dist_value(batch.obs[idx], batch.task_id)
ratio      <- exp( dist.log_prob(a) - old_logp )      # per stream / head
adv        <- (normalized) GAE advantage
pg_loss    <- clipped-surrogate( adv, ratio, clip_ratio ) # eqs 22, 31
v_loss     <- 0.5 * (value - returns)^2                # standard PPO critic
entropy    <- dist.entropy().mean()
total_loss += coeff * pg_loss + vf_coef * v_loss - ent_coef * entropy
if bc and this is the LAST (current-task) stream:      # EXPERIMENT 2 (off by default)
    total_loss += bc_coef * KL( pi_local || pi_global )
zero_grad; total_loss.backward()
clip_grad_norm( policy.parameters(), max_grad_norm )    # single global clip
opt.step()

# Continual-learning enters ONLY through the per-stream actor `coeff`; critic,
# GAE, entropy are task-agnostic PPO.

```

ProjectedAscentDual.update (the mu dual; eq 47)

```

value <- clip( value + lr*(constraint - eps), 0, max_value )
return value
# lr = eta_mu (dual step). Projection onto [0, max_value].
# The lower clip at 0 is the [.]+ projection of eq 47 (mu >= 0).
# The UPPER clip at max_value is faithful practical DEVIATION #2: the raw eq 47
# has no ceiling; the code caps mu to keep the (already normalized) dual weight
# from running away. Direction of the update is unchanged.

```

_evaluate_row(k) -- post-task greedy forgetting row

```

last <- K if cfg.eval_all_tasks else k
for i = 0 .. last-1:
    score_i, std_i <- _eval_report(global_policy, family.tasks[i])
    # GREEDY (argmax) game SCORE, pooled over fixed-seed episodes; when
    # eval_noop_enumerate is on, exact greedy via no-op enumeration. Reported
    # metric is task PERFORMANCE (raw score), NOT the discounted value V.
return row = [score_0 .. score_{last-1}], stds
# Row k of eval_matrix; the lower triangle is the forgetting matrix.

```

What is updated, and when

phi (global params)	: EVERY iteration of the global phase, by optimize_batches (ascends L_G, eqs 40, 43).
mu (dual)	: every `constraint_every` iters (slow timescale, eq 47); reset (mu <- init) at the START of each global phase.
V_k^L	: computed ONCE per cycle after the local phase, then frozen (the floor / reference; eqs 8, 18).
V_k^G	: refreshed every `constraint_every` iters -> drives shortfall.
phi^-	: set to phi* at stage end; frozen reference of the next

stage (chaining eq 6). Never stored beyond the live params.

Key intermediate quantities

V_k^L = ref_current : frozen local (current-task) value = the floor. (eqs 8,18)
 V_k^G : global's current-task value; moves during the phase. (eq 42)
shortfall = $[V_k^L - V_k^G]_+$: one-sided current-task gap (eq 26 hinge argument).
 F_k = shortfall² : one-sided squared shortfall / constraint metric. (eqs 26,48)
coeff_k = 2μ *shortfall : current-task actor-gradient weight. (eqs 38, 40)
 $\omega_i = 1/k$: past-task actor-gradient weight (eq 25 term).

Implicit assumptions

- "All previously seen environments remain available" -> FRESH old-task rollouts each iteration; no replay buffer (doc p.1, after eq 3).
- Part A stores NO expert: the current-task floor comes from a locally-trained, then frozen, throwaway π_L -- not a saved expert critic.
- Only the ACTOR is constrained; the shared-trunk critic is trained from scratch by PPO's value loss (not seeded), so V_k^G is taken from FRESH Monte-Carlo rollouts (eq 42), never read off the moving critic head.
- Uniform weights $\omega_i = 1/k$ throughout.
- Reported eval = raw GREEDY game score (task performance); the constraint uses the discounted value V .

Code <-> equation map (Updated_Objective_for_CRL.pdf, Part A)

three checkpoints $\pi_G^- \rightarrow \pi_L \rightarrow \pi_G^+$ eq 4
local init $\theta^0 = \phi^-$ eq 5
chaining $\phi^- \leftarrow \phi^*$ at stage end eq 6
score functional $J_k(\pi) = \sum \omega_i V_i$ eq 7
local objective / arg max V_k^π eqs 18, 19
local PPO gradient (clipped surrogate stand-in) ... eqs 20-23
past-task retention objective $\sum \omega_i V_i^{\phi^*}$. eq 25 (maximized)
one-sided squared hinge $F_k = [V_k^L - V_k^G]_+^2$. eq 26
constrained global problem s.t. $F_k \leq \epsilon$ eq 27
Lagrangian $L_G(\phi, \mu; \phi^-, \theta^*)$ eq 28
saddle point $\max_\phi \min_{\mu \geq 0}$ eq 29
past-task gradient $\sum \omega_i \text{grad } V_i^{\phi^*}$ eqs 33-35 (reference cancels)
differentiated hinge $-2[V_k^L - V_k^G]_+ \text{grad } V$... eqs 36-38
Lagrangian gradient $+ 2\mu [\cdot]_+ \text{grad } V$ eqs 40, 41
Monte-Carlo value / gradient estimators eq 42
primal ascent $\phi \leftarrow \phi + \beta \text{grad } L_G$ eqs 43, 44
dual ascent $\mu \leftarrow [\mu + \eta(F_k - \epsilon)]_+$ eqs 46, 47
estimated shortfall $F_k^* = [V_k^L^* - V_k^G^*]_+^2$. eq 48

Faithful practical deviations from the raw equations

- D1. Coeff-sum normalization (optimize_batches): every actor coefficient is divided by their sum before the PPO step. Direction-preserving positive rescaling; keeps the shared critic from starving under a large μ . Fixed point of eq 29 unchanged.
- D2. μ ceiling (ProjectedAscentDual): eq 47 projects only onto $\mu \geq 0$; the code also caps μ at `max_value`. Bounds the dual magnitude; update direction and the $\mu \geq 0$ projection are unchanged.